

ATLAS: Adaptive Task Offloading System in Multi-UAV-assisted Multi-Access Edge Computing

Taewon Song and Taeyoon Kim

Abstract—Multi-unmanned aerial vehicle (UAV) task offloading in edge computing environments requires coordinated decision-making under dynamic network conditions and limited computational resources. However, independently trained agents struggle to generalize across varying operational conditions, such as fluctuating UAV velocities, due to the lack of shared knowledge among agents. This paper presents an asynchronous advantage actor-critic (A3C)-based multi-UAV task offloading system that leverages parameter sharing among distributed workers to achieve superior generalization. We formulate the problem as a partially observable Markov decision process where UAV agents select optimal offloading actions—local processing, offloading, or task discard—based on computational state, queue status, and network conditions. The proposed system employs a recurrent actor-critic architecture with layer normalization to handle sequential dependencies and stabilize asynchronous training across multiple workers. Experimental evaluation against individual A3C training and independent proximal policy optimization (IPPO) demonstrates that the proposed approach achieves 29.7% higher generalization performance and 34% variance reduction over independent learning strategies. Through systematic evaluation of worker count, cloud resource availability, and gated recurrent unit (GRU) hidden state dimensions, we validate our architectural design choices and identify critical parameters that govern A3C’s advantages, providing practical guidelines for deploying distributed reinforcement learning in multi-UAV edge computing systems.

Index Terms—UAV, Task Offloading, A3C, Deep Reinforcement Learning, Multi-Agent Systems, Multi-Access Edge Computing

I. INTRODUCTION

The proliferation of unmanned aerial vehicles (UAVs) in applications ranging from surveillance to emergency response has created unprecedented opportunities for distributed edge computing. However, UAVs are inherently resource-constrained platforms: their onboard processors must handle increasingly compute-intensive workloads, such as image recognition, path planning, and sensor fusion, while preserving battery life for flight. Task offloading, in which a UAV delegates computation to a remote cloud or edge server, has emerged as a key strategy to reconcile these competing demands. As UAV deployments scale from single platforms to coordinated multi-agent systems, multi-UAV systems must make real-time offloading decisions while competing for shared cloud resources under dynamic network conditions,

making the design of effective learning algorithms increasingly critical.

While existing approaches have explored various task offloading strategies [1]–[5], a fundamental gap remains in understanding how to effectively coordinate learning among distributed UAV agents. Independent learning approaches, in which each agent optimizes its own policy in isolation, fail to leverage collective experience across agents and generalize poorly to unseen operational conditions. Recent work has shown that the update rule itself matters, e.g., independent proximal policy optimization (IPPO) can outperform independent asynchronous advantage actor-critic (A3C) without parameter sharing [6], [7], yet whether global parameter sharing provides additional benefits beyond the update rule, and under what conditions, has not been systematically studied.

Consider a disaster response scenario where multiple UAVs must process sensor data and transmit critical information to ground stations. Each UAV continuously decides whether to process tasks locally, offload them to a shared cloud server, or discard them when neither option is feasible. These decisions depend on local computational state and wireless channel conditions that can only be partially observed. Because all UAVs compete for the same shared cloud capacity, a UAV’s offloading decision directly affects the resource availability seen by its peers, making coordinated learning essential for robust system-wide performance.

In this paper, we propose ATLAS, asynchronous task offloading in multi-UAV-assisted multi-access edge computing, an A3C-based multi-UAV task offloading framework that leverages parameter sharing among distributed workers to achieve superior generalization performance across diverse operational conditions. ATLAS employs a recurrent actor-critic architecture with layer normalization to handle sequential decision dependencies and stabilize asynchronous training across multiple heterogeneous workers.

We formulate the multi-UAV task offloading problem as a partially observable Markov decision process (POMDP) where each UAV agent selects actions based on local observations. The proposed architecture integrates recurrent neural networks (RNNs) to capture temporal dependencies in cloud resource availability, which is not directly observable by individual UAVs, while layer normalization ensures stable gradient updates across asynchronously training workers. We compare ATLAS against individual A3C training and IPPO across varying worker counts, cloud resource capacities, and agent velocities to identify the operational regimes where parameter sharing provides genuine advantages over independent policies.

T. Song is with the Division of Electronics and Electrical Engineering, College of Engineering, Dongguk University, 30, Pildong-ro 1-gil, Jung-gu, Seoul, Republic of Korea (e-mail: twsong@sch.ac.kr) and T. Kim is with ... (e-mail: 2000kty@dankook.ac.kr), Corresponding author: T. Kim.

This work was supported in part by ..., and in part by ..., and in part by ...

The main contributions of this paper are as follows:

- We design and implement an A3C-based multi-UAV task offloading system with a carefully architected recurrent actor-critic network that achieves superior generalization performance through parameter sharing among distributed workers.
- We conduct systematic evaluation across worker count, cloud resource availability, and agent velocity to validate our design choices and establish practical deployment guidelines.
- We identify critical operational regimes including resource availability and worker diversity that determine whether parameter sharing provides genuine benefits over independent learning, offering practitioners clear criteria for algorithm selection in multi-UAV deployments.
- Through gated recurrent unit (GRU) hidden state analysis, we show that ATLAS implicitly encodes the unobserved shared cloud capacity, revealing the credit assignment mechanism that enables preemptive resource coordination and explaining why parameter sharing outperforms independent learners under resource scarcity.

The remainder of this paper is organized as follows. Section II reviews related work in UAV task offloading and multi-agent reinforcement learning. Section III formulates the problem as a POMDP and describes our system model. Section IV presents the proposed ATLAS architecture and the A3C-based learning algorithm. Section V evaluates the performance of the proposed approach through extensive simulations and discusses the results. Finally, Section VI concludes the paper.

II. RELATED WORKS

The intersection of UAV-assisted mobile edge computing and deep reinforcement learning (DRL) has emerged as a critical research area, with various approaches addressing the challenges of distributed task offloading optimization. This section reviews existing work across three key themes: reinforcement learning methods for task offloading, multi-agent coordination and parameter sharing strategies, and architectural components for handling partial observability in sequential decision-making systems.

A. Reinforcement Learning for UAV Task Offloading

DRL has been widely applied to UAV task offloading, with early work establishing its feasibility for single-UAV edge computing [8]. Since UAVs cannot directly observe channel conditions or remote server load, subsequent work formulated the problem as a POMDP and employed recurrent architectures, notably the deep recurrent Q-network (DRQN), to infer hidden states from observation histories [9]. As deployments scaled to multiple UAVs, multi-agent DRL (MADRL) approaches emerged in which each agent trains an independent policy [1], [2], [10], [11]. Further extensions jointly optimize offloading with trajectory planning and resource allocation [3], [4], while game-theoretic formulations address incentive design [5].

Despite these advances, existing learning-based approaches train each agent's policy independently, and game-theoretic

methods coordinate incentives rather than share learned representations. Whether global parameter sharing outperforms independent learning, and how this advantage varies with resource availability, worker diversity, and out-of-distribution deployment conditions, has not been systematically studied.

B. Distributed Learning and Parameter Sharing

A prominent approach to distributed knowledge sharing in edge computing is federated reinforcement learning, which periodically aggregates locally trained model parameters across distributed agents [12]. This paradigm has been applied to resource management in fifth-generation technology standard (5G) mobile edge computing (MEC) networks [13], cooperative edge caching for Internet of Things (IoT) systems [14], dynamic spectrum access [15], and task offloading in vehicular networks [16], [17].

While federated approaches reduce communication overhead, synchronization at fixed intervals limits how quickly collective experience propagates across agents. An alternative is A3C [18], in which workers continuously push gradient updates to a shared global model without synchronization barriers, allowing diverse experiences to propagate immediately and enabling the global model to reflect the collective behavior of all workers in real time.

A parallel line of work has questioned whether parameter sharing is strictly necessary for effective multi-agent learning. Tan [6] and Yu et al. [7] showed that independent learning with proximal policy optimization (PPO) [19] is competitive in cooperative tasks, suggesting that the update rule itself matters alongside parameter sharing.

These studies, however, focus on homogeneous, in-distribution scenarios, and how the benefit of global parameter sharing scales with worker diversity and generalizes to out-of-distribution conditions in resource-constrained UAV deployments remains unexplored.

C. Architecture Components in Deep Reinforcement Learning

Recurrent architectures have been widely adopted in reinforcement learning (RL) to address partial observability. DRQN [20] extended the deep Q-network (DQN) with a long short-term memory (LSTM) layer to maintain temporal context across steps, and subsequent work showed that GRU-based variants achieve comparable performance with lower computational overhead [21]. Mnih et al. [18] also showed that incorporating an LSTM layer into the A3C actor-critic network improves performance on partially observable tasks such as 3D maze navigation.

Beyond recurrent architectures, normalization has emerged as a key stabilizer in deep RL training. Layer normalization [22] reduces internal covariate shift across diverse input distributions, an effect particularly relevant in asynchronous multi-worker settings where gradient updates arrive from workers experiencing heterogeneous conditions. However, the interaction between recurrent components and layer normalization in multi-agent asynchronous training, including their joint impact on generalization, has not been systematically

studied. This paper addresses that gap by examining both components within A3C's parameter-sharing framework applied to multi-UAV task offloading.

III. POMDP FORMULATION

A. Problem Formulation

We formulate the multi-UAV task offloading optimization problem as a Partially Observable Markov Decision Process (POMDP), where each UAV agent operates as an intelligent decision maker in a dynamic edge computing environment. The POMDP framework enables UAV agents to make sequential decisions for incoming computational tasks while adapting to changing environmental conditions and resource availability. Furthermore, the POMDP formulation captures the inherent uncertainty in UAV operations, such as fluctuating network conditions and the computation capability of the cloud servers for task offloading.

Formally, the ATLAS framework models the task offloading problem as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the state transition function, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. Since agents cannot directly observe the full state $s \in \mathcal{S}$, each agent receives a partial observation $o \in \mathcal{O} \subseteq \mathcal{S}$ and selects an action $a \in \mathcal{A}$ according to its policy $\pi(a | o)$, receiving a reward $r_t = \mathcal{R}(s, a)$ while the environment transitions to $s' \sim \mathcal{P}(s' | s, a)$. To reason over unobserved state components, each agent maintains an internal recurrent state that serves as an implicit approximation of the belief over hidden environment variables.

The primary objective is to learn an optimal policy π^* that maximizes the expected cumulative discounted reward:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi \right]. \quad (1)$$

This formulation enables UAV agents to balance multiple competing objectives including energy efficiency, processing latency minimization, and task completion success rates in dynamic edge computing scenarios.

B. State Space

We formulate the multi-UAV task offloading problem as a partially observable Markov decision process (POMDP). In this framework, the environment's full state is objectively defined, yet each UAV agent can only perceive a subset of this state through local sensors and feedback. We define the environment state space $\mathcal{S}$ as

$$\mathcal{S} = \mathcal{O} \times \mathcal{Z} \quad (2)$$

where $\mathcal{O}$ denotes the set of **observable state components** measurable by individual UAV agents, and $\mathcal{Z}$ represents the **exogenous latent environment state components** that capture exogenous and unobservable environmental factors. While the latent components $z \in \mathcal{Z}$ are not directly included in the agent's input vector, they are essential to preserve the Markov property of the underlying system dynamics.

1) *Observable State Components*: The observable state space $\mathcal{O}$ consists of system information directly measurable by each UAV agent and is defined as

$$\mathcal{O} = S_{local} \times S_{context} \times S_{mec}. \quad (3)$$

Local Device State (S_{local}): This component characterizes the UAV's local computational resources, task queue, and processing feedback:

$$S_{local} = C_l \times E_r \times C_q \times T_q \times I_l \times I_o \quad (4)$$

where C_l represents a set of available computation units, E_r denotes a set of remaining epochs during one episode, C_q and T_q capture sets of queue computation units and processing times, and I_l, I_o provide binary success indicators for local and offload operations. Above-mentioned values are normalized within $[0, 1]$ to ensure stable neural network training and effective learning across different operational scales.

Environmental Context ($S_{context}$): This encompasses dynamic environmental conditions directly measurable by the UAV:

$$S_{context} = V \times C \quad (5)$$

where V represents normalized velocity context, and C indicates normalized local computation capacity context. We set the minimum and the maximum velocities to 30 km/h and 100 km/h [23], respectively, and the computation capacity ranges from 0 to 200 units. Both V and C are normalized to $[0, 1]$ according to their respective marginal values.

MEC Server State (S_{mec}): This component captures the observable state of nearby, or physically attached mobile edge computing servers through feedback mechanisms:

$$S_{mec} = C_m \times T_m \quad (6)$$

where $C_m = \prod_{k \in K} C_{m,k}$ represents ongoing computation units for each task $k \in K$, and $T_m = \prod_{k \in K} T_{m,k}$ denotes Corresponding processing times for each task $k \in K$. UAV agents can observe MEC state through acknowledgement messages and status updates from edge infrastructure.

2) *Exogenous Latent State Components*: The exogenous latent state space $\mathcal{Z}$ captures environment dynamics that influence system performance but remain outside the direct sensing range of individual UAVs, and is defined as

$$\mathcal{Z} = S_{cloud} \times S_{channel} \quad (7)$$

where S_{cloud} represents the remote cloud server state and $S_{channel}$ represents the wireless channel state between each UAV and the infrastructure.

Cloud Server State (S_{cloud}): This component models the remote cloud infrastructure availability in the core network, which is hidden from direct UAV observation because of the aggregated nature of cloud resources serving numerous MEC servers simultaneously:

$$S_{cloud} = C_c \times T_c \quad (8)$$

where $C_c = \prod_{k \in K} C_{c,k}$ represents ongoing computation units for each task $k \in K$, and $T_c = \prod_{k \in K} T_{c,k}$ denotes processing times for each task $k \in K$. The shared and distributed architecture of cloud infrastructure, coupled with the large

number of attached MEC nodes, makes direct state monitoring infeasible from individual UAV perspectives.

Channel State ($S_{channel}$): This captures the wireless communication quality between UAV and infrastructure as

$$S_{channel} = Q \quad (9)$$

where $Q \in \{0, 1\}$ denotes binary channel quality indicators, with 0 and 1 representing bad and good channel states, respectively. The channel state follows a finite-state Markov channel (FSMC) formulation, adopting a two-state representation derived from the Gilbert–Elliott model [24], [25] for Rayleigh fading channels [26].

Although the hidden state components $\mathcal{Z}$ are not included in the UAVs' local observations, they are explicitly incorporated into the environment state to ensure Markovian dynamics. UAV agents execute their policies based solely on observable states in $\mathcal{O}$, resulting in a POMDP formulation. The RNN architecture employed in this work is designed to mitigate the effects of partial observability at the agent level by exploiting temporal correlations in observable state sequences, rather than explicitly reconstructing the hidden environment state.

C. Action Space

The action space $\mathcal{A}$ in the ATLAS framework defines the discrete set of decisions available to each UAV agent when processing incoming computational tasks. We formulate the action space as a finite discrete set:

$$\mathcal{A} = \{0, 1, 2\} \quad (10)$$

where 0, 1, and 2 stand for each defined action, respectively. A specific procedure will be stated as follows.

- **Local Processing ($a = 0$):** When this action is selected, the UAV processes the computational task using its associating MEC server. The local processing action triggers immediate computation on the UAV, consuming local computational resources but avoiding transmission delays and potential network failures.
- **Offloading ($a = 1$):** This action involves transmitting the computational task to available cloud servers for remote processing. The offloading action incurs transmission energy costs and may introduces network-dependent failures, but potentially reduces local computational load and enables processing of resource-intensive tasks.
- **Task Discard ($a = 2$):** When system conditions are unfavorable for both local processing and offloading, because of insufficient local resources, saturated server queues, or poor channel conditions that prevent successful task completion, the agent may choose to discard the task. Task discard minimizes immediate resource consumption but results in task failure penalties in the reward function.

D. State Transition Function

The state transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ characterizes the probabilistic evolution of the environment state in response to the agent's actions. Following the state definition in Section III-A, the environment state is given by

$s = (o, z)$ where $o \in \mathcal{O}$ denotes the observation state available to UAV agents, and $z \in \mathcal{Z}$ represents hidden exogenous environmental states. We also define s' notation to represent the state of the next time epoch, e.g., $s' = (o', z')$.

Although components of z , i.e., cloud information and channel quality, evolve on time scales that are not synchronized with the UAV decision process, the state transition function is defined with respect to decision epochs rather than physical time. Specifically, the value of each exogenous component is sampled at each decision epoch and incorporated into the environment state, ensuring that the transition $P(s' | s, a)$ is well-defined and Markovian with respect to the decision-aligned state.

The complete transition function can be decomposed as

$$P(s' | s, a) = P(o' | o, z, a) \cdot P(z' | z), \quad (11)$$

where the observable and exogenous components evolve according to distinct mechanisms. This decomposition reflects the fact that UAV actions directly influence task execution, queue dynamics, and local resource utilization, while the evolution of cloud and channel states is governed by external processes independent of individual UAV decisions.

1) *Observable State Transitions:* The state transitions of $P(o' | o, z, a)$ are governed by the agent's action selection and are fully observable by the UAV. These include resource consumption, queue management, and action outcome indicators. In contrast to hidden state transitions, observable components evolve deterministically given the action and current resource availability. $P(o' | o, z, a)$ can be represented as follows:

$$\begin{aligned} P(o' | o, z, a) = & P(c'_l | c_l, c_q, t_q, c_m, t_m, a) \\ & \times P(e'_r | e_r) \\ & \times P(c'_q) \\ & \times P(t'_q) \\ & \times P(i'_l | i_l, c_l, c_q, c_m, a) \\ & \times P(i'_o | i_o, c_q, q, c_c, a) \\ & \times P(c'_m | c_l, c_q, t_q, c_m, t_m, a) \\ & \times P(t'_m | c_l, c_q, t_q, c_m, t_m, a), \end{aligned} \quad (12)$$

where $c_l \in C_l$, $e_r \in E_r$, $c_q \in C_q$, $t_q \in T_q$, $i_l \in I_l$, $i_o \in I_o$, $c_m \in C_m$, and $t_m \in T_m$ denote the realizations of the corresponding state variables, and their primed counterparts represent the values at the next decision epoch.

Local Computation Units (c_l): Available local computation units evolve through resource consumption upon successful local processing and recovery upon MEC task completion. Define the local processing feasibility indicator:

$$\phi_l \triangleq \mathbf{1}[c_l \geq c_q] \cdot \mathbf{1}[\exists k \in \{1, \dots, K\} : c_{m,k} = 0] \cdot \mathbf{1}[c_q > 0], \quad (13)$$

where $\mathbf{1}[\cdot]$ denotes the indicator function, yielding 1 when the enclosed condition holds and 0 otherwise. This indicator means that $\phi_l = 1$ indicates that local processing is feasible: sufficient local resources are available, the MEC queue has capacity, and a pending task exists.

Let $\kappa_m = \min\{k : c_{m,k} = 0\}$ denote the first available MEC queue slot. The intermediate MEC state after task enqueue is

$$\begin{aligned}\tilde{c}_{m,k} &= c_{m,k} + c_q \cdot \mathbf{1}[a=0] \cdot \phi_l \cdot \mathbf{1}[k=\kappa_m], \\ \tilde{t}_{m,k} &= t_{m,k} + t_q \cdot \mathbf{1}[a=0] \cdot \phi_l \cdot \mathbf{1}[k=\kappa_m].\end{aligned}\quad (14)$$

Then the transition of c_l is given by

$$\begin{aligned}P(c'_l | c_l, c_q, t_q, c_m, t_m, a) \\ = \mathbf{1}\left[c'_l = c_l - c_q \cdot \mathbf{1}[a=0] \cdot \phi_l + \sum_{k=1}^K \tilde{c}_{m,k} \cdot \mathbf{1}[\tilde{t}_{m,k} = 1]\right].\end{aligned}\quad (15)$$

The first negative term captures resource consumption when a task is successfully scheduled for local processing, while the summation captures resource recovery when MEC-scheduled tasks complete their processing.

Remaining Epochs (e_r): The remaining epoch counter decrements deterministically at each decision epoch, independent of the agent's action:

$$P(e'_r | e_r) = \mathbf{1}[e'_r = e_r - 1] \quad (16)$$

Queue Task Arrivals (c_q, t_q): A new computational task arrives at each decision epoch with computation units and processing time drawn independently from discrete uniform distributions:

$$\begin{aligned}P(c'_q | \cdot) &= \frac{1}{C_{\max}}, \quad \forall c'_q \in \{1, \dots, C_{\max}\}, \\ P(t'_q | \cdot) &= \frac{1}{T_{\max}}, \quad \forall t'_q \in \{1, \dots, T_{\max}\}.\end{aligned}\quad (17)$$

where $C_{\max}$ and $T_{\max}$ denote the maximum number of computation units and the maximum processing time, and are predefined as 200 and 50, respectively. These arrivals are mutually independent and independent of all other state variables, modeling the unpredictable workload in UAV task offloading scenarios.

Success Indicators (i_l, i_o): Each indicator is updated only when the corresponding action is taken and retains its previous value otherwise. The offload feasibility additionally depends on hidden state components $q \in \mathcal{Z}$ and $c_c \in \mathcal{Z}$. Define the offload feasibility indicator:

$$\phi_o \triangleq \mathbf{1}[\exists k : c_{c,k} = 0] \cdot \mathbf{1}[c_q > 0] \cdot \mathbf{1}[Q = 1] \cdot \mathbf{1}[c_c^{\text{avail}} \geq c_q], \quad (18)$$

where c_c^{avail} denotes the remaining shared cloud computation capacity across all UAV agents. Then, the indicator transitions are as follows:

$$P(i'_l | i_l, c_l, c_q, c_m, a) = \mathbf{1}[i'_l = \phi_l \cdot \mathbf{1}[a=0] + i_l \cdot \mathbf{1}[a \neq 0]], \quad (19)$$

and

$$P(i'_o | i_o, c_q, q, c_c, a) = \mathbf{1}[i'_o = \phi_o \cdot \mathbf{1}[a=1] + i_o \cdot \mathbf{1}[a \neq 1]]. \quad (20)$$

MEC Server State (c_m, t_m): The MEC queue undergoes a three-phase update per decision epoch: enqueue, completion removal, and processing-time decrement. Using the intermediate queue state $(\tilde{c}_{m,k}, \tilde{t}_{m,k})$ defined in (14), tasks satisfying

$\tilde{t}_{m,k} = 1$ are identified as completed; their computation units contribute to local resource recovery as described in the c'_l transition. The final MEC state after completion removal and one-epoch decrement is

$$P(c'_{m,k} | c_l, c_q, t_q, c_m, t_m, a) = \mathbf{1}[c'_{m,k} = \tilde{c}_{m,k} \cdot \mathbf{1}[\tilde{t}_{m,k} \neq 1]], \quad (21)$$

and

$$P(t'_{m,k} | c_l, c_q, t_q, c_m, t_m, a) = \mathbf{1}[t'_{m,k} = \max(\tilde{t}_{m,k} - 1, 0)], \quad (22)$$

for each $k \in \{1, \dots, K\}$, after which active tasks are compacted to the leading positions of the queue vector.

Environmental Context (v, c): Each UAV agent may have a distinct velocity and maximum computation capacity, reflecting heterogeneous platform characteristics. These parameters are sampled once at episode initialization and remain constant throughout the episode:

$$P(v' | v) = \mathbf{1}[v' = v], \quad (23)$$

and

$$P(c' | c) = \mathbf{1}[c' = c]. \quad (24)$$

2) *Hidden State Transitions:* The hidden state space $\mathcal{Z} = S_{\text{cloud}} \times S_{\text{channel}}$ captures exogenous environment dynamics that are not directly observable by individual UAV agents. While these components influence state transitions and rewards, their evolution is not synchronized with the agent-level decision epochs.

Among the hidden components, the channel state $q \in Q$ evolves as an autonomous finite-state Markov process. The cloud state $c_c \in C_c$ and $t_c \in T_c$ follows queueing dynamics analogous to those of MEC servers; however, unlike MEC states, cloud queue information is not directly observable due to aggregation and shared access across multiple UAV agents. Moreover, the evolution of c_c and t_c is affected by successful offloading actions from other UAVs, rendering its dynamics asynchronous with respect to the local decision process of any individual agent.

Accordingly, the hidden-state transition at each decision epoch is modeled as a snapshot of these exogenous processes, and is given by

$$P(z' | z) = P(q' | q) \cdot P(c'_c, t'_c | c_c, t_c), \quad (25)$$

where the cloud-state transition is treated as an exogenous stochastic process independent of the current agent's action.

Channel State (q): The binary channel quality evolves as a two-state finite-state Markov channel (FSMC) based on the Gilbert–Elliott model [24], [25], with transition rates governed by the Doppler effect [26].

The Doppler frequency is given by

$$f_d = \frac{v \cdot f_0}{c_{\text{light}}}, \quad (26)$$

where $f_0 = 5.06$ GHz is the carrier frequency of permitted for UAV by Federal Communications Commission (FCC) [27] and c_{light} is the speed of light. The transition probabilities are

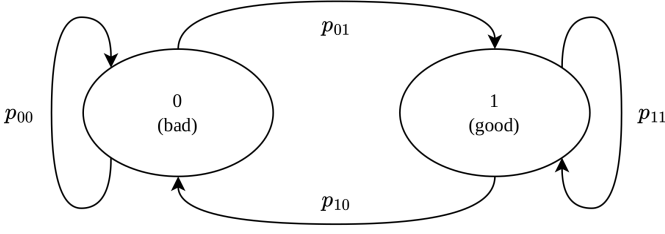


Fig. 1: Transmission probability of the Gilbert–Elliot model represented as a two-state Markov diagram. The Good state denotes a high-quality channel allowing successful transmission, whereas the Bad state represents a lower-quality channel.

parameterized by the normalized Doppler spread $\bar{f}_d = f_d \cdot T_p$, where T_p denotes the decision epoch duration:

$$P(q' | q) = \begin{cases} p_{01}, & q = 0, q' = 1, \\ p_{00} = 1 - p_{01}, & q = 0, q' = 0, \\ p_{10}, & q = 1, q' = 0, \\ p_{11} = 1 - p_{10}, & q = 1, q' = 1, \end{cases} \quad (27)$$

where each transition probabilities are as

$$\begin{aligned} p_{01} &\triangleq \frac{\bar{f}_d \sqrt{2\pi \gamma_{\text{thr}}/\bar{\gamma}}}{\exp(\gamma_{\text{thr}}/\bar{\gamma}) - 1}, \\ p_{10} &\triangleq \bar{f}_d \sqrt{2\pi \gamma_{\text{thr}}/\bar{\gamma}}, \end{aligned} \quad (28)$$

where γ_{thr} and $\bar{\gamma}$ denote the signal-to-noise ratio (SNR) threshold and average SNR, respectively. Both transition rates increase with velocity through $\bar{f}_d$, reflecting increased channel variability at higher UAV speeds.

The corresponding two-state Markov representation of the Gilbert–Elliot channel model is illustrated in Fig. 1, where the channel alternates between Good and Bad states according to the transition probabilities p_{01} and p_{10} .

Cloud State (c_c, t_c): The cloud state captures the aggregated task queue dynamics in the core network, which are not directly observable by individual UAV agents. Unlike MEC server states, the cloud queue evolves asynchronously due to background traffic and successful offloading actions from multiple UAVs, rendering its dynamics exogenous from the perspective of any single agent.

To model temporal correlations in cloud workload, the cloud state (c_c, t_c) is represented as a finite-state Markov process, where c_c and t_c denote the normalized cloud processing capacity and service latency levels, respectively. At each decision epoch, the agent observes a snapshot of this process through its effect on task completion outcomes.

E. Reward Function

The reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ quantifies the total computation successfully completed at each decision epoch. No reward is assigned at the moment of action selection; reward is received only when previously scheduled tasks finish processing at either the MEC server or the cloud. This deferred-reward structure requires the agent to implicitly

associate its current scheduling decisions with future task completions.

A task is identified as completed when its remaining processing time reaches unity in the intermediate queue state. The reward is the total computation units recovered from all completed tasks across both processing locations, scaled by a constant ρ :

$$\begin{aligned} \mathcal{R}(s, a) \\ = \rho \left(\underbrace{\sum_{k=1}^K \tilde{c}_{m,k} \cdot \mathbf{1}[\tilde{t}_{m,k} = 1]}_{R_{\text{MEC}}} + \underbrace{\sum_{k=1}^K \tilde{c}_{c,k} \cdot \mathbf{1}[\tilde{t}_{c,k} = 1]}_{R_{\text{cloud}}} \right), \end{aligned} \quad (29)$$

where $(\tilde{c}_{m,k}, \tilde{t}_{m,k})$ is the intermediate MEC queue state defined in the observable state transitions in (14). The cloud intermediate state $(\tilde{c}_{c,k}, \tilde{t}_{c,k})$ follows the same enqueue structure with action $a=1$ and feasibility indicator ϕ_o in place of $a=0$ and ϕ_l :

$$\tilde{c}_{c,k} = c_{c,k} + c_q \cdot \mathbf{1}[a=1] \cdot \phi_o \cdot \mathbf{1}[k=\kappa_c], \quad (30)$$

and

$$\tilde{t}_{c,k} = t_{c,k} + t_q \cdot \mathbf{1}[a=1] \cdot \phi_o \cdot \mathbf{1}[k=\kappa_c], \quad (31)$$

where $\kappa_c = \min\{k : c_{c,k} = 0\}$ denotes the first available cloud queue slot.

The MEC term R_{MEC} captures completions at the agent-controlled server, while the cloud term R_{cloud} captures completions at the shared cloud infrastructure. Both terms contribute identically to the reward, reflecting that the optimization objective is to maximize total computation throughput regardless of processing location. The scaling factor ρ controls the reward magnitude relative to the policy gradient updates and is treated as a hyperparameter.

IV. DEEP REINFORCEMENT LEARNING ALGORITHM WITH A3C ARCHITECTURE

The ATLAS framework addresses the UAV task offloading problem as a POMDP in which agents cannot directly observe channel quality or cloud resource availability. To handle this partial observability, we employ a recurrent actor-critic network with GRU-based hidden state propagation.

A. Recurrent A3C Architecture

The proposed agent employs a recurrent actor-critic architecture designed to handle the partial observability inherent in UAV environments. As illustrated in Fig. 2, the network consists of three functional stages: an observation embedding block, a recurrent processing unit, and twin output heads for policy and value estimation.

The input layer receives both the current environmental observation and the RNN hidden state carried over from the previous decision epoch. The embedding block first transforms the raw observation into a high-dimensional feature vector, utilizing layer normalization to stabilize the feature distribution before recurrent processing. Subsequently, a gated recurrent

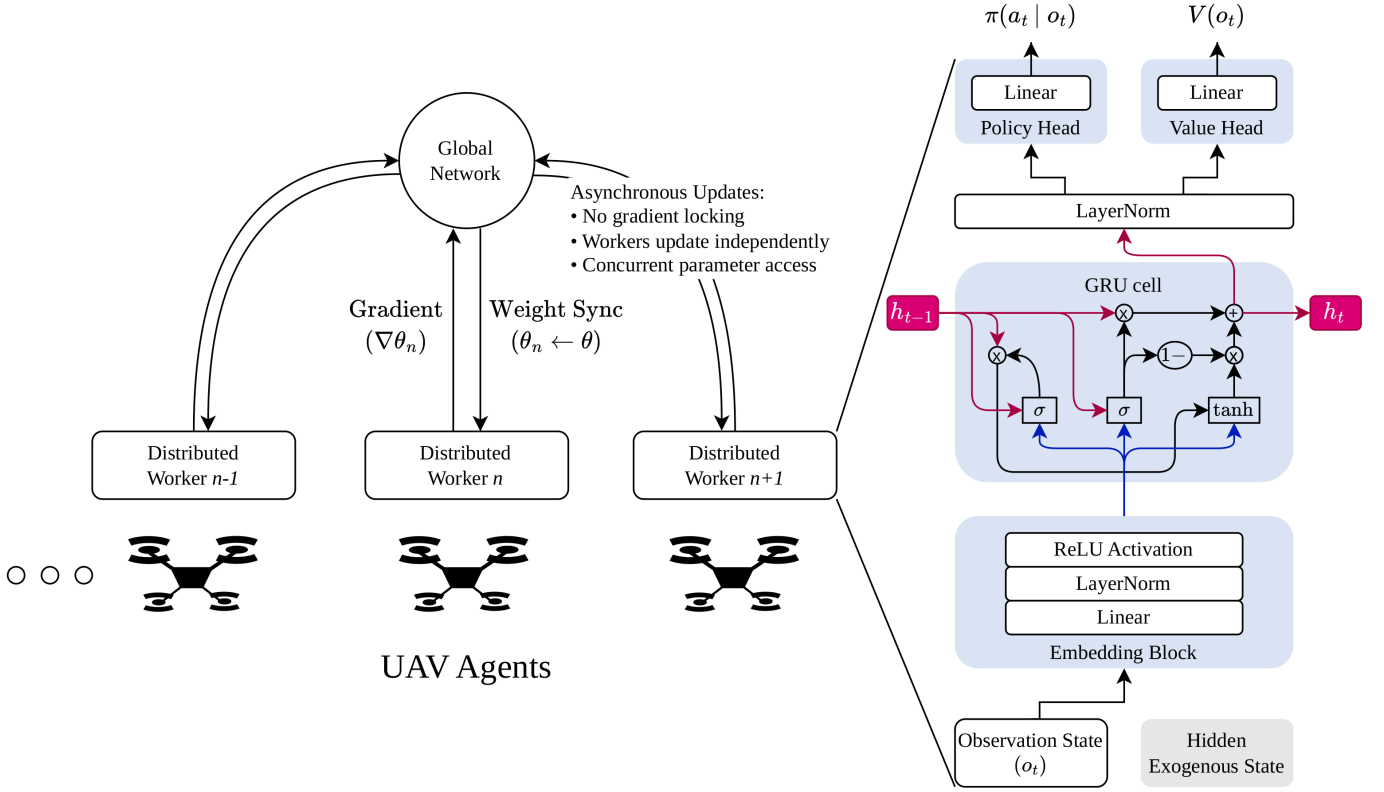


Fig. 2: Proposed recurrent actor-critic architecture in the A3C framework. Observations are encoded by an embedding block and processed by a GRU-based recurrent module to capture temporal dependencies, after which separate heads produce the policy and value estimates. σ and $1-$ stand for the sigmoid activation function and complement operation in GRU gates, respectively.

unit (GRU) integrates these features with the previous hidden state. This recurrent mechanism effectively maintains a temporal bottleneck that captures sequential dependencies in wireless channel quality and cloud workload, which are part of the exogenous latent state.

Finally, the integrated features are mapped through two separate linear heads. The policy head produces a categorical distribution over the available actions, while the value head provides a scalar estimate of the state value. This design enables the agent to develop an internal representation of unobservable environmental factors, facilitating robust decision-making across heterogeneous operational scenarios. All internal layers utilize orthogonal initialization and layer normalization to ensure stable gradient updates during the asynchronous training process.

B. Learning Algorithm

ATLAS deploys W parallel workers, each maintaining a local copy of the recurrent actor-critic network, and coordinates their learning through asynchronous gradient updates to a shared global model.

A3C Global Training: W workers run in parallel, each assigned a distinct agent velocity. All workers asynchronously update a shared global model through lock-free gradient synchronization. At each update, a worker (i) copies the current global parameters, (ii) collects a T -step rollout, (iii) computes

gradients on its local copy, and (iv) asynchronously applies those gradients to the global model without waiting for other workers. The worker then re-synchronizes with the updated global parameters.

Individual Training: Each worker maintains and trains its own model independently, with no parameter or gradient sharing. Training proceeds through standard rollout collection, gradient computation, and local parameter updates. Workers synchronize only at episode boundaries to ensure comparable cloud resource conditions across training paradigms.

Both paradigms minimize the following combined loss over a T -step rollout:

$$\mathcal{L} = \underbrace{-\frac{1}{T} \sum_{t=0}^{T-1} \hat{A}_t \log \pi(a_t | o_t; \theta)}_{\mathcal{L}_{\text{policy}}} + \underbrace{\beta_v \frac{1}{T} \sum_{t=0}^{T-1} (V(o_t; \theta) - G_t)^2}_{\mathcal{L}_{\text{value}}} - \underbrace{\beta_H \frac{1}{T} \sum_{t=0}^{T-1} H(\pi(\cdot | o_t; \theta))}_{\text{entropy bonus}}, \quad (32)$$

where $\hat{A}_t = G_t - V(o_t; \theta)$ is the advantage estimate, normalized to zero mean and unit variance over the rollout, and $G_t = \sum_{\tau=t}^{T-1} \gamma^{\tau-t} r_\tau + \gamma^{T-t} V(o_T; \theta)$ is the bootstrapped T -step return. Gradients are clipped to a maximum ℓ_2 -norm of $g_{\max}$.

Algorithm 1 Training phase of ATLAS using A3C with RNN

```

1: Initialize global actor-critic network  $\theta$  with recurrent architecture
2: Initialize shared optimizer and cloud resource state  $\mathcal{R}_{\text{cloud}}$ 
3: for each worker  $w \in \{1, \dots, W\}$  in parallel do
4:   Copy global parameters:  $\theta_w \leftarrow \theta$ 
5:   for episode  $e = 1$  to  $E_{\text{max}}$  do
6:     Initialize observation  $o_0$ 
7:     RNN hidden state  $h_0 \leftarrow \mathbf{0}$ 
8:     while episode not terminated do
9:       // Collect T-step rollout
10:      for  $t = 0$  to  $T - 1$  do
11:        Compute  $\pi(\cdot \mid o_t, h_t; \theta_w)$  and  $V(o_t, h_t; \theta_w)$ 
12:        Sample  $a \sim \pi(\cdot \mid o_t, h_t; \theta_w)$ 
13:        Execute  $a$ , observe  $r_t$  and  $o'$ 
14:        Update  $h'$  via GRU; if done,  $h' \leftarrow \mathbf{0}$ 
15:      end for
16:      // Compute returns and advantages
17:      if not done then
18:         $R \leftarrow V(o_T, h_T; \theta_w)$ 
19:      else
20:         $R \leftarrow 0$ 
21:      end if
22:      for  $t = T - 1$  down to  $0$  do
23:         $R \leftarrow r_t + \gamma R$ ;  $A_t \leftarrow R - V(o_t, h_t; \theta_w)$ 
24:      end for
25:      // Calculate loss
26:      Compute loss according to (32)
27:      // Asynchronous update
28:      Compute gradients  $\nabla_{\theta_w} \mathcal{L}$  and clip
29:      Copy gradients to global:  $\nabla_{\theta} \leftarrow \nabla_{\theta_w}$ 
30:      Update global:  $\theta \leftarrow \theta - \alpha \nabla_{\theta}$ 
31:      Synchronize local:  $\theta_w \leftarrow \theta$ 
32:    end while
33:    if early stopping condition is met then
34:      break
35:    end if
36:  end for
37: end for
38: return Trained global network  $\theta$ 

```

Algorithm 1 details the complete training procedure. A critical implementation aspect is RNN hidden state management: workers initialize $h_0 = \mathbf{0}$ at each episode start and reset hidden states to zero upon terminal transitions, ensuring gradient estimates remain uncontaminated across episode boundaries. The asynchronous updates enable workers to operate independently, improving training efficiency through continuous parallel exploration.

V. PERFORMANCE EVALUATION

For performance evaluation, we conduct extensive simulations with Python 3.10 along with PyTorch [28], which is an open-source deep learning framework, and multiprocessing library for parallel worker execution. The UAV environment is implemented using standard gymnasium-style application pro-

TABLE I: Simulation Parameters

Parameter	Value
<i>Network Hyperparameters</i>	
Hidden dimension	128
RNN type	GRU (1 layer)
Learning rate α	10^{-4}
Discount factor γ	0.99
Entropy coefficient β_H	0.05
Value loss coefficient β_v	0.25
Gradient clipping g_{max}	2.0
Rollout length T	20
Reward scale ρ	0.05
<i>Environment Parameters</i>	
Number of workers W	5 (10, 20)
Episodes per worker	2000
Max computation units (local)	200
Max computation units (cloud)	1000 (500, 2000)
Max episode length	100
Task queue size	20
Agent velocities (training)	5–25 km/h
Agent velocities (testing)	5–100 km/h

gramming interface (API) [29] with custom reward functions and state representations tailored for task offloading scenarios.

The simulation environment models $W = 5, 10, 20$ UAV agents operating in a mobile edge computing scenario with shared cloud resources. Each agent is configured with the following key parameters: maximum local computation units of 200, shared cloud capacity of 1000 units, maximum episode length of 100 time steps, and task queue size of 20. Agent velocities are distributed across the range of 5–25 km/h during training to represent diverse mobility patterns.

All models are trained for 2000 episodes per worker using the combined loss function defined in (32). The discount factor γ is set to 0.99, entropy coefficient β_H to 0.05, and value loss coefficient β_v to 0.25. The learning rate is initialized to 10^{-4} with gradient clipping at maximum ℓ_2 -norm of 2.0. For the recurrent architecture, we employ a single-layer GRU with hidden dimension of 128, combined with layer normalization for training stability. Network hyperparameters and environment parameters are summarized in Table I.

We compare two training paradigms under identical environment conditions to isolate the effect of parameter sharing. ATLAS employs W parallel workers that asynchronously update a shared global model through lock-free gradient synchronization, as detailed in Algorithm 1. Each worker maintains a local copy of the network, collects T -step rollouts with $T = 20$, computes gradients, and asynchronously applies them to the global parameters without waiting for other workers. This enables continuous parallel exploration across diverse agent velocities while sharing learned representations. The global model aggregates experience from all workers, potentially improving generalization across varying network conditions.

As a baseline for comparison, individual training, specifically, individual advantage actor-critic (A2C) assigns each worker its own independent model without any parameter or gradient sharing. Each worker independently optimizes its local model using the same A2C loss function and hyperparameters as ATLAS, but applies gradients only to its own model. Workers synchronize only at episode boundaries to en-

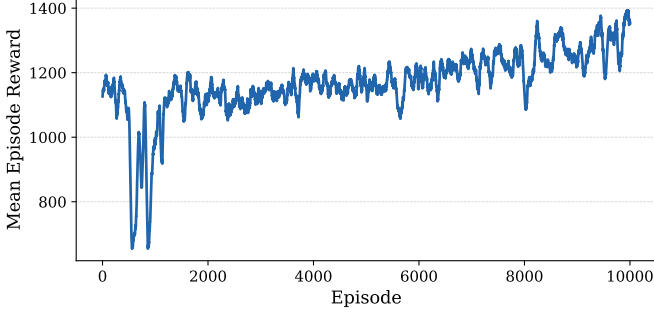


Fig. 3: Training convergence of ATLAS over 10,000 episodes per worker. The solid curve shows the 50-episode smoothed mean return averaged across all workers; the dashed curves denote the ± 1 standard deviation band (smoothed) and the gray trace is the raw per-episode mean.

sure fair comparison under identical cloud resource dynamics. This paradigm represents conventional single-agent actor-critic training scaled to multiple agents without knowledge transfer.

As a second baseline, we include Independent PPO (IPPO) [6], [7], which applies the proximal policy optimization algorithm [19] independently to each worker without parameter sharing. Each worker maintains its own actor-critic network and updates it using the clipped surrogate objective with clip ratio $\epsilon = 0.2$ and $K = 4$ epochs per rollout. Like Individual training, IPPO workers do not share gradients or model parameters; the key difference is the use of PPO’s clipped update instead of the A3C-style advantage actor-critic loss. IPPO represents the state-of-the-practice in independent multi-agent learning and serves to isolate the benefit of parameter sharing from the choice of policy gradient algorithm.

The key distinction lies in parameter sharing: ATLAS aggregates gradients from diverse experiences across all workers into a single shared model, while Individual and IPPO training each maintain W separate models that learn independently. All three paradigms use identical network architectures and environment configurations, differing only in the training algorithm and the presence or absence of parameter sharing across workers.

A. Training Convergence of ATLAS

Fig. 3 illustrates the training convergence of ATLAS over 10,000 episodes per worker with parallel workers, each assigned a distinct agent velocity in the range 5–25 km/h. The learning curve exhibits three characteristic phases. In the initial phase within episodes 0–500, the smoothed return starts near 1,100 but briefly dips to approximately 800, reflecting transient instability from asynchronous gradient updates. This is because all workers simultaneously push gradients to the global model from slightly stale snapshots, temporarily pulling parameters in conflicting directions.

Following this adjustment, the return recovers steadily to approximately 1,100–1,200 by episode 2,000, indicating that the asynchronous dynamics stabilize as workers collectively converge toward a consistent global policy. Beyond episode 2,000, the smoothed return exhibits a gradual upward trend

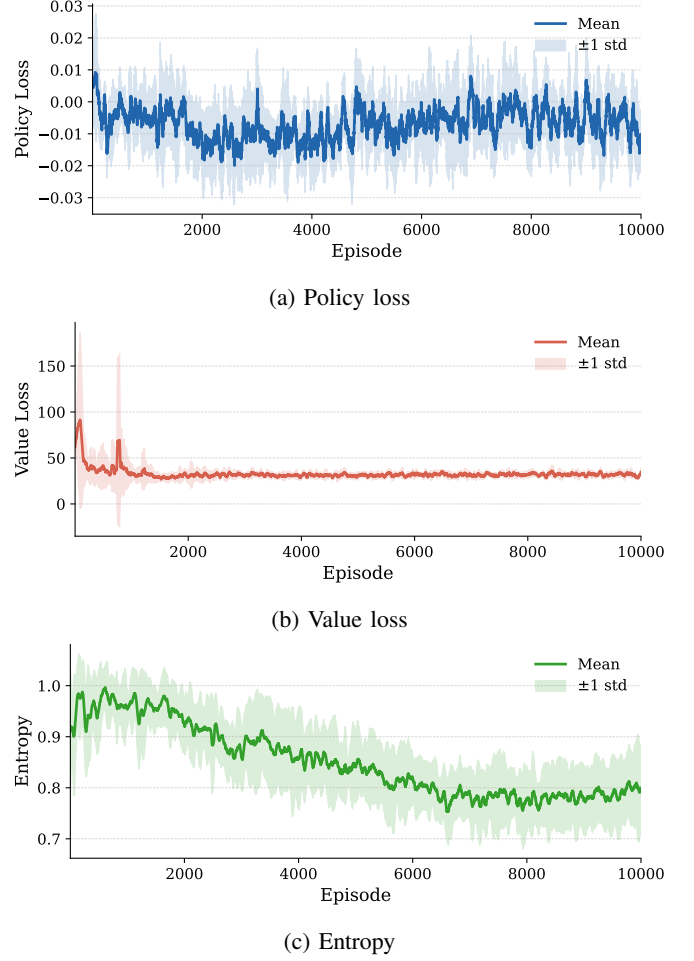


Fig. 4: Loss components of ATLAS during training (mean ± 1 std over 6 independent runs, smoothed with a 50-episode window, $\rho = 0.05$).

from approximately 1,100 to 1,300, reflecting continued incremental improvement as the global model accumulates diverse gradient signals from workers operating at distinct velocities. Taken together, Fig. 3 confirms that ATLAS reaches a stable operating regime within 2,000 episodes, which justifies the training budget adopted in subsequent experiments defined in Table I.

Fig. 4 shows the evolution of the three loss components of ATLAS, incorporating A3C, as defined in (32) during training. Note that the logged value loss and entropy are the unweighted quantities $(V(o_t; \theta) - G_t)^2$ and $H(\pi(\cdot | o_t; \theta))$, respectively, before multiplication by β_v and β_H ; the actual contributions to the total loss are $\beta_v \mathcal{L}_{\text{value}}$ and $\beta_H \mathcal{L}_{\text{entropy}}$ as in (32). The policy loss in Fig. 4a oscillates around zero throughout training, confirming that the actor maintains a consistent policy under the asynchronous update scheme. The value loss in Fig. 4b decreases sharply from an initial peak of approximately 80–180 to a steady level near 30, indicating that the critic progressively improves its return estimates as the global model accumulates experience across diverse velocity conditions. The entropy in Fig. 4c starts near 1.0 and gradually declines to approximately 0.80 by episode 10,000, reflecting a

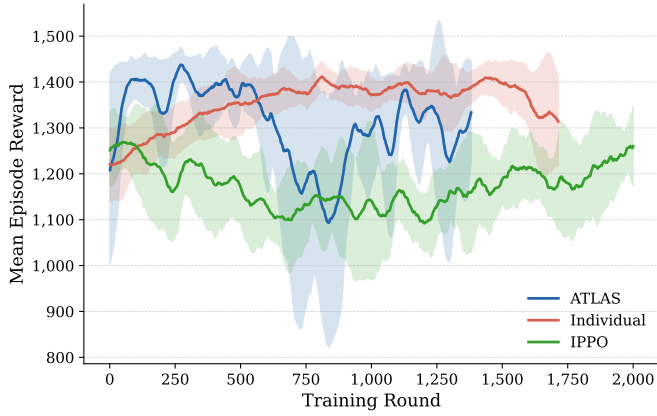


Fig. 5: Training reward curves of ATLAS, Individual, and IPPO with $W = 5$ workers, each assigned a distinct velocity in 5–25 km/h. Solid lines and shaded bands represent the mean and ± 1 standard deviation across ten independent seeds, smoothed with a 50-episode window.

steady shift toward more decisive action selection as the policy matures. This monotonic decline is consistent with the entropy regularization term $\beta_H = 0.05$ in the loss function, which penalizes overly deterministic policies but permits entropy to decrease as the policy converges. The stochastic nature of the environment also contributes: since channel quality fluctuates randomly at each step, the near-optimal policy is inherently mixed between local processing and offloading depending on instantaneous channel conditions, rather than being fully deterministic.

B. Training Performance

Fig. 5 shows the mean episode reward during training across ten independent seeds. In the early episodes, ATLAS exhibits visible fluctuations that are absent in the Individual and IPPO curves. This transient instability arises from the asynchronous update mechanism, that is, all workers simultaneously push gradients to the global model, and gradients computed from slightly stale model snapshots can transiently pull the global parameters in conflicting directions. Individual workers, each updating only their own model, are insulated from this cross-worker interference and therefore exhibit smoother early-training curves. IPPO workers are similarly independent but show moderate variance throughout training, reflecting the sensitivity of the clipped surrogate objective to rollout-level policy drift across workers.

Despite these differences in early-training dynamics, all three methods converge to comparable reward levels. Training employs an early-stopping criterion that halts execution after a minimum of 1,000 episodes once three conditions are simultaneously met: 1) the 100-episode moving-average reward stabilizes within 0.5% relative change, 2) the greedy evaluation return shows no improvement over 12 consecutive evaluation intervals, and 3) the policy entropy change falls below 3%. The difference in final training reward between ATLAS and Individual is not statistically significant, confirming that parameter sharing does not confer a decisive advantage within

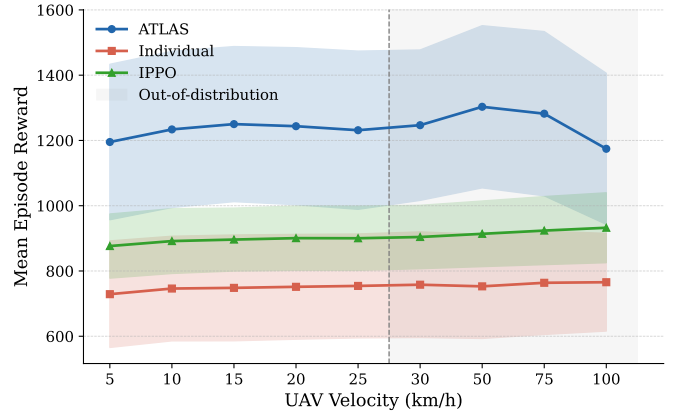


Fig. 6: Generalization performance across agent velocities (5–100 km/h) with $W = 5$ workers. The shaded region indicates the out-of-distribution range (> 25 km/h). Shaded bands denote ± 1 standard deviation across 10 seeds.

the narrow training distribution of 5–25 km/h. IPPO achieves a slightly lower final reward than Individual, consistent with the conservative step-size constraint of the clipped surrogate objective. The decisive performance differences among the three methods emerge only under out-of-distribution conditions, as we examine in Subsection. V-C.

C. Generalization Performance Across Agent Velocities

The critical distinction among the training paradigms emerges under out-of-distribution (OOD) deployment conditions. To assess velocity generalization, we evaluate the trained models across a velocity range of 5–100 km/h without any fine-tuning. ATLAS uses a single shared global model, and its training reward is reported as the mean across all five workers. Individual workers are each trained at a fixed velocity (5, 10, 15, 20, or 25 km/h). Velocities 5–25 km/h are in-distribution; 30–100 km/h are OOD. For Individual and IPPO, each of the five worker models is independently evaluated across all nine test velocities, which are 5, 10, 15, 20, 25, 30, 50, 75, and 100 km/h, and the reported generalization reward is the mean over all five workers.

Fig. 6 shows the mean episode reward as the agent velocity is swept from 5 to 100 km/h for all three methods, with per-worker and per-velocity breakdowns provided in Table II. This evaluation uses a greedy policy on fully trained models, isolating the generalization capability of each paradigm from training-time exploration behavior.

The comparison reveals two distinct and separable effects. First, IPPO outperforms Individual training across the entire velocity range, demonstrating that the PPO update rule itself, through clipped surrogate objectives and multiple update epochs per rollout, yields more stable per-agent policies even in the absence of any parameter sharing. Second, ATLAS consistently outperforms IPPO by a similar margin, demonstrating an additional and independent contribution from parameter sharing. Crucially, this advantage is not confined to the training distribution: ATLAS maintains its lead over IPPO in both the in-distribution range and the OOD range, indicating that

TABLE II: Generalization reward of each worker model evaluated independently across all nine test velocities (5–100 km/h) from async training (mean over 10 seeds). Velocities 5–25 km/h are in-distribution; 30–100 km/h are out-of-distribution (OOD).

Model	Worker	Trained Vel. (km/h)	In-Distribution Gen. Reward (km/h)					OOD Gen. Reward (km/h)			
			5	10	15	20	25	30	50	75	100
ATLAS	Global	5–25	1195.2	1233.8	1250.1	1243.7	1231.4	1246.8	1303.2	1282.0	1174.5
Individual A2C	W0	5	907.8	921.8	916.6	923.7	925.0	940.5	945.9	952.9	932.7
	W1	10	786.0	813.4	831.7	841.4	843.1	850.8	875.8	942.0	984.8
	W2	15	627.0	653.4	653.9	649.0	657.7	657.4	612.9	591.3	587.1
	W3	20	614.7	622.3	613.6	616.7	614.1	620.3	618.6	632.4	641.4
	W4	25	709.1	720.2	725.6	726.6	731.2	720.9	711.4	700.6	681.8
IPPO	W0	5	1077.4	1112.1	1113.5	1115.7	1114.7	1119.1	1089.3	1087.2	1070.6
	W1	10	726.0	729.1	744.0	751.3	752.6	763.9	833.5	930.9	1006.2
	W2	15	919.0	924.9	931.9	928.8	929.1	936.1	928.8	913.2	904.0
	W3	20	911.4	929.4	938.1	939.0	946.4	943.2	1008.0	999.2	1020.2
	W4	25	748.2	762.7	753.7	768.3	758.7	758.2	709.8	688.4	661.4

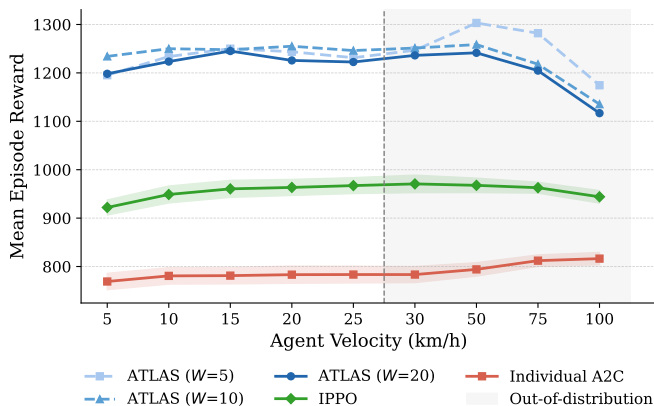


Fig. 7: Impact of worker count $W \in \{5, 10, 20\}$ on generalization performance over 10 seeds. Reward curves for ATLAS, IPPO, and Individual across agent velocities 5–100 km/h; the dotted vertical line separates in-distribution from OOD velocities.

parameter sharing generalizes to unseen conditions rather than merely memorizing the training velocities. Together, these two effects are approximately additive, suggesting that the update rule and parameter sharing address complementary aspects of generalization.

D. Impact of System Parameters

1) *Effect of Worker Count*: We evaluate the effect of the worker count $W \in \{5, 10, 20\}$ on generalization performance by running each configuration with 10 independent seeds across the full velocity range of 5–100 km/h.

Fig. 7 illustrates the worker-count effect. ATLAS maintains a consistently high reward level across all three configurations, while IPPO and Individual training remain substantially lower and show no consistent trend with W .

In principle, increasing W broadens the velocity distribution covered during training, since each worker is assigned a distinct speed and all workers share a global model. However, in the asynchronous training setting, a larger worker

pool also introduces greater gradient interference: workers simultaneously push updates computed from slightly stale model snapshots, and conflicting gradient directions can partially offset the benefit of wider coverage. As a result, the marginal improvement from adding workers is modest, and ATLAS’s generalization reward remains roughly stable across $W \in \{5, 10, 20\}$.

By contrast, the performance gap between ATLAS and the two baselines is pronounced and stable across all worker counts. IPPO and Individual A2C, which lack any mechanism for cross-worker knowledge transfer, cannot exploit the additional velocity diversity introduced by more workers, and their performance remains flat regardless of W . This confirms that the advantage of parameter sharing is not contingent on a specific worker count, but is a structural property of the shared-model architecture that persists even when asynchronous gradient interference partially limits the scaling benefit.

2) *Cloud Resource Availability*: We examine how the total cloud computation budget $C \in \{500, 1000, 2000\}$ units affects generalization performance across the full velocity range of 5–100 km/h over 10 independent seeds for ATLAS, IPPO, and Individual A2C. Since the cloud resource pool is shared among all $W = 5$ workers, these configurations correspond to an effective per-worker allocation of approximately 100, 200, and 400 units, respectively.

Fig. 8 reveals a striking dependence on resource scarcity. Under the most constrained budget, that is, $C = 500$, ATLAS’s advantage over Individual A2C is largest, and the gap over IPPO is also clearly visible. As the budget increases to the baseline and the abundant setting, $C = 1000$ and $C = 2000$, respectively, absolute rewards rise across all methods, but the gap between ATLAS and Individual A2C narrows monotonically, while the gap between ATLAS and IPPO remains relatively stable across all three conditions.

This systematic decay of performance gaps reflects the role of resource coordination in shared-pool environments. Under severe resource constraints, every offloading decision is in direct competition with other workers’ demands. Policies that can implicitly coordinate by reserving cloud capacity for high-workload situations and deferring when collective demand

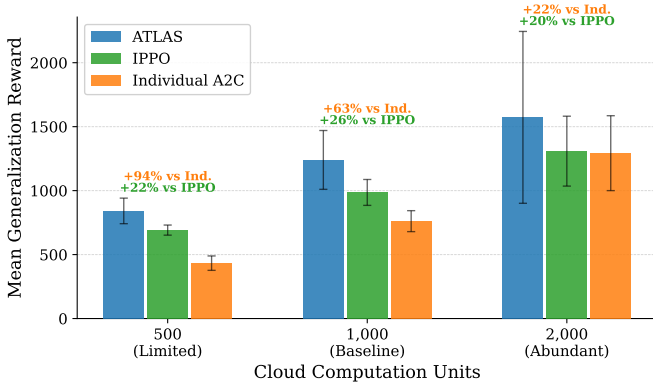


Fig. 8: Generalization performance across cloud computation capacities $C \in \{500, 1000, 2000\}$ for ATLAS, IPPO, and Individual A2C (10 seeds for ATLAS; 5 seeds for IPPO and Individual A2C). Error bars denote one standard deviation. Percentages above each group indicate ATLAS's relative improvement over IPPO (green) and Individual A2C (orange).

is high dominate those that optimize greedily in isolation. ATLAS's global parameter sharing enables workers to collectively internalize the shared-resource constraint: gradients from workers experiencing resource contention flow into the global model alongside those from workers with spare capacity, producing a cooperative allocation policy across the entire worker pool. Individual A2C, which updates each agent's parameters without access to other workers' experiences, cannot resolve this multi-agent credit assignment problem, and suffers the largest deficit under scarcity.

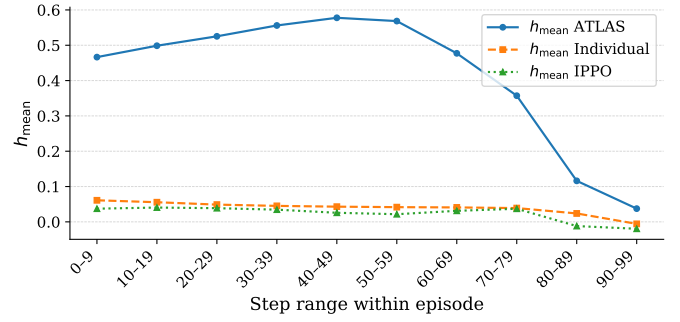
IPPO achieves an intermediate position at $C = 500$: its clipped surrogate objective constrains the magnitude of each update, preventing the aggressive cloud over-exploitation that destabilizes Individual A2C. However, this benefit is transient. At $C = 2000$, demand saturation eliminates the coordination bottleneck, and IPPO converges to near-parity with Individual A2C, confirming that IPPO's advantage under low resources is scarcity-driven rather than structural. ATLAS retains a consistent advantage over IPPO across all resource levels, attributable to the continued benefit of global gradient aggregation across heterogeneous velocity conditions.

Together, these results demonstrate that ATLAS's generalization advantage is not a fixed margin but a resource-sensitive property: parameter sharing is most beneficial precisely when shared resources create the greatest need for coordinated allocation.

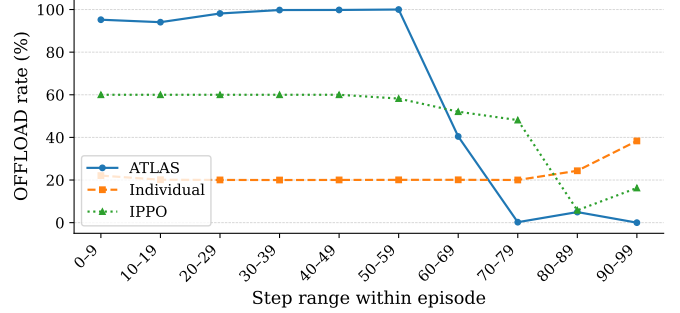
E. Analysis of Implicit Coordination

The performance patterns observed in the preceding subsections raise a mechanistic question: how does ATLAS learn to coordinate resource allocation without explicit communication between workers? To answer this, we examine what information the GRU hidden state encodes during inference, and how it shapes action selection throughout an episode.

Fig. 9 traces the within-episode dynamics of the GRU hidden state and offloading behavior across all three methods. Here, $h_{\text{mean}}(t) = \frac{1}{d} \sum_{i=1}^d [\mathbf{h}_t]_i$ denotes the mean activation



(a) GRU h_{mean} per step range.



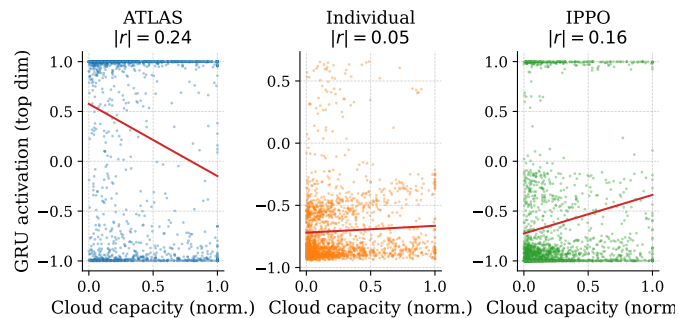
(b) OFFLOAD rate per step range.

Fig. 9: Within-episode dynamics of GRU h_{mean} (top) and OFFLOAD rate (bottom) for ATLAS, Individual, and IPPO, averaged across 100 episodes per worker (5 workers, $v = 15$ km/h).

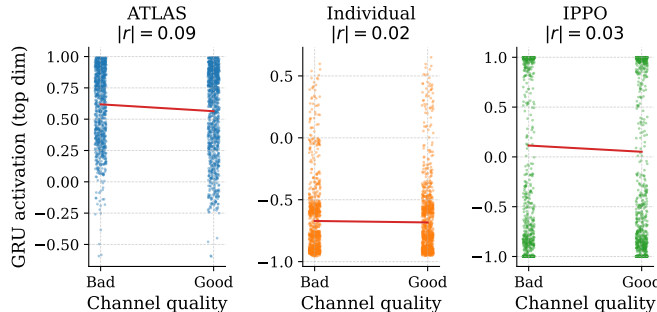
across all $d = 128$ GRU hidden units at time step t , where $\mathbf{h}_t \in \mathbb{R}^d$ is the GRU hidden state vector that encodes the agent's internal summary of the observation history accumulated up to step t , as shown in Fig. 2. This scalar projection provides a compact view of the overall recurrent state dynamics without requiring visualization of all individual dimensions. At episode onset (steps 0–9), cloud capacity is near full. ATLAS's h_{mean} rises gradually from 0.47 to 0.58 as cloud resources deplete (steps 0–50), then collapses abruptly once the cloud is exhausted (steps 60–70), accompanied by a sharp drop in OFFLOAD rate from $\sim 100\%$ to near zero. This behavioral transition is not driven by any observable signal; cloud capacity is absent from the agent's observation, so the shift must be inferred entirely from the accumulated history of offload outcomes encoded in the recurrent state.

In contrast, the h_{mean} of both Individual and IPPO remains flat throughout the episode, indicating that neither method develops a dynamic internal representation of the shared cloud state. While IPPO also reduces its OFFLOAD rate at steps 60–70, this response is reactive, triggered by explicit offload failures observed at the current step, rather than predictive. ATLAS, by contrast, begins modulating its hidden state well before cloud exhaustion (steps 0–50), suggesting anticipatory resource management.

To confirm that ATLAS's cloud tracking is encoded in the GRU rather than derived from observable inputs, we computed the Pearson correlation between each of the 128 GRU dimensions and the cloud capacity that is unobserved



(a) GRU activation (top correlated dimension per method) vs. normalized cloud capacity.



(b) GRU activation (top correlated dimension per method) vs. channel quality.

Fig. 10: Pearson correlation between the most responsive GRU dimension of each method and two unobserved environment variables, normalized cloud capacity (top) and channel quality (bottom), evaluated with 5 concurrent workers sharing a cloud pool ($v = 15$ km/h). Red lines show linear fits; r values are reported per method.

by the workers across all time steps in Fig. 10. The most correlated dimension of ATLAS reaches $|r| = 0.24$, compared to $|r| = 0.05$ for Individual A2C and $|r| = 0.16$ for IPPO (Fig. 10a). Crucially, no method encodes channel quality ($|r| \leq 0.10$ for all methods, Fig. 10b). Under the two-state Markov channel model with the FCC-permitted UAV carrier frequency of $f_0 = 5.06$ GHz, the mean sojourn time per state ranges from approximately 4 steps at 25 km/h, meaning the channel typically transitions 25 times within a single 100-step episode. This rapid alternation provides no persistent trend for the GRU to encode: unlike cloud capacity, which depletes monotonically over the episode and rewards long-horizon recurrent tracking, channel quality reverts direction too frequently to leave a stable imprint on the hidden state.

These per-dimension correlations are consistent with the h_{mean} results but reveal an important distinction for IPPO. IPPO's top-correlated dimension reaches $|r| = 0.16$, below ATLAS ($|r| = 0.24$) but substantially above Individual A2C ($|r| = 0.05$), suggesting that a small number of GRU dimensions do encode cloud-related information. However, IPPO's h_{mean} remains flat, as shown in Fig. 9a, indicating that this encoding is confined to a few isolated dimensions while the majority of the hidden state is unresponsive to cloud depletion. In ATLAS, cloud tracking is distributed broadly

enough across the 128-dimensional hidden state that the mean activation itself drifts with resource availability. This sparse-vs.-distributed encoding distinction explains why IPPO can observe individual offload failures but cannot form the pool-wide resource estimate that enables ATLAS's anticipatory behavior.

The divergence between ATLAS and the other methods in h_{mean} drift is attributable to parameter sharing. Cloud capacity is a global resource shared among all workers; its depletion rate therefore reflects the collective offload behavior of the entire worker pool, not any single agent's actions. Individual and IPPO workers optimize independent policies, so each agent's recurrent state is informed only by its own offload history, which is a noisy, partial signal of the global resource state. ATLAS workers, by contrast, update a single shared GRU: gradients from all workers, including those encountering offload failures due to contention, are aggregated into one model. This allows ATLAS to form a coherent, pool-wide estimate of cloud availability, and to preemptively shift toward local processing before exhaustion forces reactive adaptation. This predictive coordination is the credit assignment mechanism that Individual and IPPO cannot replicate, and it explains why ATLAS's advantage over both benchmarks is most pronounced precisely when cloud resources are scarce, as described in Section V-D2.

VI. CONCLUSION

This paper presented ATLAS, an A3C-based multi-UAV task offloading framework in which parameter sharing among distributed workers drives robust generalization across heterogeneous operating conditions. We formulated the problem as a POMDP and proposed a recurrent actor-critic architecture with GRU and layer normalization to handle partial observability and stabilize asynchronous training.

Simulations against Individual training and IPPO revealed complementary contributions of the update rule and parameter sharing: the PPO objective improves generalization by +20.2% over individual training, while ATLAS's parameter sharing adds a further +37.1% over IPPO. This advantage is most pronounced under resource scarcity and amplifies with worker count. GRU hidden state analysis confirmed the mechanism: ATLAS's recurrent state encodes an implicit estimate of the unobserved shared cloud capacity, enabling preemptive offloading decisions before resource exhaustion, whereas Individual and IPPO workers respond only reactively.

One limitation identified by the hidden state analysis is the absence of temporal channel encoding: no method correlates its GRU state with channel quality ($|r| \leq 0.09$), because channel fluctuations are near-memoryless at the training velocity range. At higher UAV speeds or under correlated fading conditions (e.g., Markov-modulated channels, spatial shadowing), exploiting temporal channel structure could further improve scheduling performance. Future work should explore auxiliary channel-prediction objectives or attention-based temporal encoding to extend ATLAS's cooperative resource management to channel-aware coordination.

REFERENCES

- [1] A. M. Seid, G. O. Boateng, B. Mareri, G. Sun, and W. Jiang, "Multi-agent drl for task offloading and resource allocation in multi-uav enabled iot edge network," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4531–4547, 2021.
- [2] H. Guo, Y. Wang, J. Liu, and C. Liu, "Multi-uav cooperative task offloading and resource allocation in 5g advanced and beyond," *IEEE Transactions on Wireless Communications*, vol. 23, no. 1, pp. 347–359, 2024.
- [3] H. Hao, C. Xu, W. Zhang, S. Yang, and G.-M. Muntean, "Joint task offloading, resource allocation, and trajectory design for multi-uav cooperative edge computing with task priority," *IEEE Transactions on Mobile Computing*, vol. 23, no. 9, pp. 8649–8663, 2024.
- [4] B. Li, R. Yang, L. Liu, J. Wang, N. Zhang, and M. Dong, "Robust computation offloading and trajectory optimization for multi-uav-assisted mec: A multiagent drl approach," *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4775–4786, 2024.
- [5] M. Wang, L. Zhang, P. Gao, X. Yang, K. Wang, and K. Yang, "Stackelberg-game-based intelligent offloading incentive mechanism for a multi-uav-assisted mobile-edge computing system," *IEEE Internet of Things Journal*, vol. 10, no. 17, pp. 15 679–15 689, 2023.
- [6] C. S. De Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. Torr, M. Sun, and S. Whiteson, "Is independent learning all you need in the starcraft multi-agent challenge?" *arXiv preprint arXiv:2011.09533*, 2020.
- [7] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in neural information processing systems*, vol. 35, pp. 24 611–24 624, 2022.
- [8] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning for uav-enabled wireless communications with edge computing," *IEEE Transactions on Cognitive Communications and Networking*, vol. 7, no. 1, pp. 131–141, 2020.
- [9] T. Song, "Drqn-based task offloading in uav-assisted mobile edge computing environments with hidden channel conditions," in *2024 15th International Conference on Information and Communication Technology Convergence (ICTC)*, 2024, pp. 2153–2154.
- [10] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in uav-assisted mobile edge computing," *IEEE Transactions on Wireless Communications*, vol. 21, no. 9, pp. 6949–6962, 2022.
- [11] S. Guo, Z. Li, J. Yan, and X. Zhao, "Cognitive uav-assisted offloading for mobile edge computing based on multi-agent deep reinforcement learning," in *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, 2025, pp. 1–6.
- [12] H. H. Zhuo, W. Feng, Q. Xu, Q. Yang, and Y. Lin, "Federated reinforcement learning," *CoRR*, vol. abs/1901.08277, 2019. [Online]. Available: <http://arxiv.org/abs/1901.08277>
- [13] S. Yu, X. Chen, Z. Zhou, X. Gong, and D. Wu, "When deep reinforcement learning meets federated learning: Intelligent multitimescale resource management for multiaccess edge computing in 5g ultradense network," *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 2238–2251, 2021.
- [14] X. Wang, C. Wang, X. Li, V. C. M. Leung, and T. Taleb, "Federated deep reinforcement learning for internet of things with decentralized cooperative edge caching," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9441–9455, 2020.
- [15] P. Tehrani, F. Restuccia, and M. Levorato, "Federated deep reinforcement learning for the distributed control of nextg wireless networks," in *2021 IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN)*, 2021, pp. 248–253.
- [16] X. Zhao, Y. Wu, T. Zhao, F. Wang, and M. Li, "Federated deep reinforcement learning for task offloading and resource allocation in mobile edge computing-assisted vehicular networks," *Journal of Network and Computer Applications*, vol. 229, p. 103941, 2024.
- [17] S. Moon and Y. Lim, "Federated deep reinforcement learning based task offloading with power control in vehicular edge computing," *Sensors*, vol. 22, no. 24, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/24/9595>
- [18] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," *International conference on machine learning*, pp. 1928–1937, 2016.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [20] M. J. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable mdps," in *AAAI fall symposia*, vol. 45, 2015, p. 141.
- [21] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *arXiv preprint arXiv:1412.3555*, 2014.
- [22] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.
- [23] 3GPP, "TS 22.125: Unmanned Aerial System (UAS) Support in 3GPP," ETSI, Sophia Antipolis, France, Tech. Rep., 2022.
- [24] E. N. Gilbert, "Capacity of a Burst-noise Channel," *Bell system technical journal*, vol. 39, no. 5, pp. 1253–1265, 1960.
- [25] E. O. Elliott, "Estimates of Error Rates for Codes on Burst-noise Channels," *The Bell System Technical Journal*, vol. 42, no. 5, pp. 1977–1997, 1963.
- [26] Q. Zhang and S. A. Kassam, "Finite-state Markov Model for Rayleigh Fading Channels," *IEEE Transactions on communications*, vol. 47, no. 11, pp. 1688–1692, 1999.
- [27] "FCC Adopts Initial Rules for Uncrewed Aircraft Use in the 5 GHz Band." [Online]. Available: <https://www.fcc.gov/document/fcc-adopts-initial-rules-uncrewed-aircraft-use-5-ghz-band>
- [28] "PyTorch." [Online]. Available: <https://pytorch.org>
- [29] "Gymnasium Documentation." [Online]. Available: <https://gymnasium.farama.org>